Geavanceerde computerarchitectuur: lab-4

Lukas Dewachter

February 2024

1 Learning objective

In this lab session we learn about the different types of memory in CUDA. We compare these and then check what works in certain situations. The second part of the lab is looking at the difference in coalesced memory and uncoalesced memory acces.

2 Exercise 1

In this exercise we will convert an image into a greyscale image. We will use two methods: coalesced and uncoalesced. Coalesced memory access means that the memory we are accessing is next to each other and thus easier to consecutively access. We will then compare the results to see if they match our predictions.



2.1 Results

As we can see in the results the coalesced method is always better than the uncoalesced method. Then we also see that the execution speeds lower if we increase the amount of threads per block but once we have 1024 threads per block its marginally slower again. This can be because of Thread divergence. If we need one more thread at the end of execution we need a new 1024 block of threads instead of a smaller group which could explain the delay. Another reason could be the startup time of the blocks taking longer because of the size.



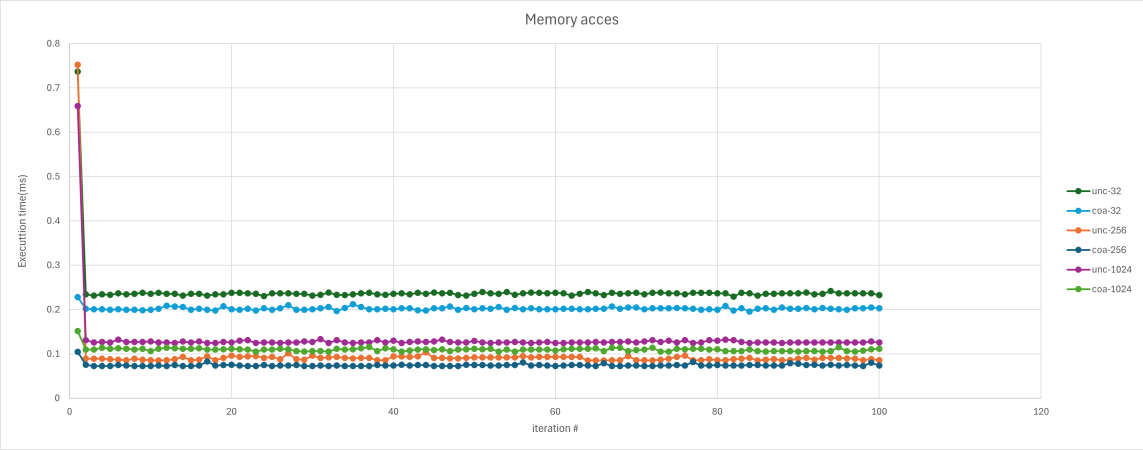


Figure 1: executing speeds for coalesced and uncoalesced memory at different thread block sizes

3 Exercise 2

This exercise will be about different memory types. We will use matrix multiplication to test which one works fastest. We will be using global memory like we did in all previous labs this is accessible by all threads in the grid and the CPU. It is very large compared to the other memory types but it's also slow because it needs to be able to read/write and be accessible by a lot of elements. Shared memory is memory stored inside a block of threads and has a low access latency but not a lot of storage space. Constant memory is accessible by all threads and the CPU but it's read only which means it will have a very low access latency.

3.1 results

Constant is the fastest but doesn't allow for much data. The memory allocated for constant memory was full on my pc after using a 30X30 matrix. Shared memory was expected to be fastest because it's on chip but in this case it's not (i used more than 1000 executions and tried multiple times). Shared memory should be the main workhorse when the data becomes big enough.

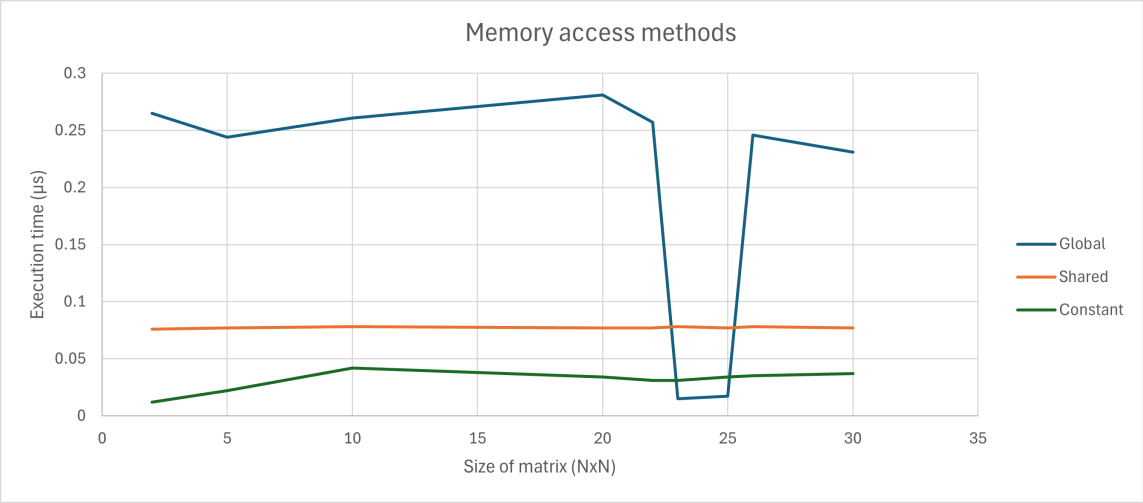


Figure 2: executing speeds for global vs shared vs constant memory

4 Conclusions

When trying to optimise your code you should see if you can use coalesced memory accesses to get more performance out of your code. Then for the data storage in the GPU you should use constant for small elements and shared memory for big elements.